An Introduction to Lout

James Clements III

An Introduction to Lout

A six-slide tour of the document formatting system

James Clements III — May 2026

What is Lout?

- A high-level **document formatting language** written in ANSI C
- Authored by Jeffrey H. Kingston (University of Sydney, 1991)
- Reads a stream of `@`-prefixed expressions, emits PostScript or PDF
- ~36k lines of C; one binary, no runtime dependencies
- Distributed under the GPL; current upstream is **3.43**

The Galley Model

Lout's central abstraction is the **galley**: a stream of typeset material that flows into a *target*. Unlike TeX's box-and-glue model, galleys are *lazy* and *recursive* — a footnote galley can spawn its own footnotes.

The cost: a less mature ecosystem. The benefit: mixing music, math, diagrams, and prose in one source is uniquely natural.

A Touch of Math

A galley is a partial function, and Lout's break algorithm minimises a penalty over candidate break-point sequences. The penalty is the sum, over all lines, of the *badness* of each line plus a stretch term proportional to the square of its stretch ratio.

(Inline KaTeX-rendered math on slides is intentionally omitted in this deck — the SVG fragments produced by `@Math` need extra plumbing on the `slidesf` flow path. Use plain prose, or render the equation in an external tool and `![](img.svg)` it instead.)

Hello, Lout

The canonical first program is just four logical lines:

- `@SysInclude { doc }` — pull in the standard document setup
- `@Doc @Text @Begin` — open the body
- `@PP` followed by `Hello, world.` — one paragraph
- `@End @Text` — close the body

Build it with `lout hello.lt > hello.ps`.

(A verbatim code block isn't safe inside `@Overhead` — Lout's `@Verbatim` doesn't escape the literal `@End` token, so a closing `@End @Text` inside the listing ends up closing the slide instead. Use prose, or a screenshot, for source-code examples on slides.)

Package Anatomy

- `doc` — ordinary documents (the default)
- `report` — papers with cover sheet, abstract, table of contents
- `book` — multi-chapter books with running heads
- `slides` — overhead transparencies (what you are looking at)
- `eq` — mathematical equations
- `tab` — tables
- `diag` — diagrams, flowcharts, and trees

(Tables on slides currently trip a `coltex` symbol-table issue in this fork's `slidesf`; restrict tables to `type: doc` / `report` for the moment.)

The Pipeline

The **mdlout** toolchain, four hops from source to output:

Markdown --> Lout --> PostScript --> PDF

(`@Diag` works fine in `type: doc` and `type: report` — on slides it currently collides with `slidesf`'s symbol table, so this deck substitutes a plain centred-display arrow chain.)

Further Reading

- *Lout: A Reader's Guide*, Kingston (2013)
- The `doc/user/` tree in the lout source — nearly 200 pages
- William Chia-Wei Cheng's william8000 fork on GitHub
- `mdlout` README in this repository for Markdown extensions

Thank you.